

OAuth 2.0 Grant Types – In-Depth Engineering Guide

Executive Summary: This report provides an exhaustive, low-level analysis of OAuth 2.0 grant types and tokens for an engineering audience. We cover all major OAuth grants – Authorization Code (with and without PKCE), Client Credentials, Refresh Token, Device Authorization, Implicit (deprecated) and Token Exchange – explaining when and how to use each. The focus is on **Authorization Code flow internals**: detailed HTTP messages, code generation and storage, front-channel vs back-channel steps, PKCE, state/nonce, one-time code binding, and all security protections (CSRF, replay attacks, etc). We include sequence diagrams (Mermaid) for each flow, concrete HTTP examples, pseudocode, tables, and sample schemas. We also cover token creation (JWT vs opaque), signing (JWS algorithms), JWT claims, JWKS public keys, token lifecycles, refresh-token rotation/revocation/introspection, and common errors (`invalid_grant`, `redirect_uri_mismatch`, etc). Official sources (RFC 6749, RFC 7636, RFC 8628, RFC 8693, OIDC Core Spec, Ping documentation) are cited throughout for authoritative detail. The target reader is an experienced developer wanting full protocol knowledge.

OAuth 2.0 Roles and Flow Types

Roles:

- **Resource Owner (User):** Person with an account.
- **Client:** Application (web, mobile, backend) requesting access.
- **Authorization Server (AS):** Issues tokens (e.g. Ping Federate).
- **Resource Server (API):** Accepts access tokens to serve resources.

Grant Categories:

- **3-Legged Grants (User Involved):** User must log in/consent. Includes *Authorization Code*, *Implicit (legacy)*, *Device Code*.
- **2-Legged Grants (No User):** Just the client authenticates. Includes *Client Credentials*.
- **Refresh Token:** A special grant to obtain new access tokens (no user action).
- **Token Exchange (RFC 8693):** Swap one token for another in advanced delegation scenarios.

Typical Uses:

- **Web or Mobile Apps:** Authorization Code (with PKCE for public clients).
- **Server-to-Server (Microservices):** Client Credentials.
- **Long-lived Sessions:** Refresh Tokens.
- **Browserless Devices:** Device Authorization (user enters code elsewhere).
- **Legacy SPAs:** (Implicit – *not recommended*) ¹.

Below table summarizes each grant's key attributes:

Grant Type	Flow Type	User Involved	Tokens Issued	When to Use
Auth Code (with PKCE)	3-legged	Yes	<i>Authorization Code</i> → Access & ID & (optional) Refresh	Web apps, mobile apps (with PKCE for public clients) ²
Client Credentials	2-legged	No	Access Token only	Service-to-service, backend microservices
Refresh Token	N/A	No	New Access Token (+ new refresh)	Renew access tokens without user login
Device Code	3-legged	Yes (via another device)	Access Token (+ Refresh)	Smart TVs, IoT devices (no built-in browser)
Implicit (legacy)	3-legged	Yes	Access Token (ID Token)	<i>Old SPAs – avoid!</i> ¹
Token Exchange	N/A (extension)	Yes/No	Another token	Advanced delegation (e.g. act-on-behalf-of)

Each of these flows is diagrammed and explained in detail below.

Authorization Code Grant (3-Legged)

The **Authorization Code Grant** is the core OAuth flow for user login on web or mobile apps ². It has two main phases: a **front-channel** (browser redirect) and a **back-channel** (server-to-server).

Sequence (Mermaid Diagram)

```
sequenceDiagram
    participant User as Resource Owner (Browser)
    participant Client as Client Application
    participant AS as Authorization Server (Ping)
    participant API as Resource Server

    User ->> Client: Open app / click login
    Client -->> User: Redirect user to AS /authorize\n(with client_id, redirect_uri, state, PKCE)
    User ->> AS: GET /authorize?response_type=code&client_id=...&redirect_uri=...&scope=...&state=...&code_challenge=...
    AS -->> User: (Prompt login & consent)
    User ->> AS: [Submit credentials/MFA]
    AS -->> User: 302 Redirect to client callback\nwith `?code=AUTH_CODE&state=xyz`
```

```

User ->> Client: Redirect URL loaded (with code)
Client ->> AS: POST /token\n(grant_type=authorization_code, code=AUTH_CODE,
redirect_uri=..., client_id, client_secret, code_verifier)
AS ->> AS: Validate code, client, PKCE, redirect_uri; mark code used
AS -->> Client: { access_token: "...", id_token: "...", refresh_token: "...",
expires_in:... }
Client ->> API: GET /resource (Authorization: Bearer access_token)
API ->> API: Validate token, scopes, etc.
API -->> Client: 200 OK (resource data)

```

Front-Channel (User's Browser):

1. **Authorization Request:** Client redirects user agent to the AS's `/authorize` endpoint, with query parameters:

- `response_type=code` (indicating Authorization Code grant) ³ .
- `client_id` (registered identifier) ³ .
- `redirect_uri` (registered callback URL) ⁴ .
- `scope` (requested permissions).
- `state` (opaque value for CSRF protection) ⁵ .
- **PKCE parameters** (for public clients): `code_challenge` and `code_challenge_method` ⁶ .

Example GET:

```

GET /authorize?response_type=code
&client_id=client123
&redirect_uri=https%3A%2F%2Fapp.example.com%2Fcb
&scope=openid%20email%20profile
&state=xyzABC
&code_challenge=QwErTy...
&code_challenge_method=S256 HTTP/1.1
Host: auth.example.com

```

1. **User Login & Consent:** The AS authenticates the user (login form, MFA) and asks for consent to grant the requested scopes.
2. **Authorization Response (Code Issuance):** If the user approves, the AS **generates an authorization code** (a short-lived random string) and redirects the browser to `redirect_uri` with `?code=...&state=...` .

Example redirect:

```

HTTP/1.1 302 Found
Location: https://app.example.com/cb?code=AbCdEfGhIj&state=xyzABC

```

- The AS includes the original `state` to tie the response to the request ⁷ . - **Code Lifetime:** The code must expire quickly (RFC6749 recommends ≤ 10 minutes, typically 30–60 sec) ⁸ . - **One-Time Use:** Each code is valid for a single exchange; reuse yields error. - **Storage:** Internally, AS stores a record linking `code`

→ (client_id, redirect_uri, user_id, scope, expiry, used=false, code_challenge if PKCE). For example:

code	client_id	user_id	redirect_uri	scope	expires_at	used	code_challenge	method
AbCdEfGhIj	client123	user789	https://app.example.com/cb	openid email profile	2026-04-27T12:53:45Z	false	QwErTyUiOp...	S256

- **DB vs Cache:** Implementations may use a fast in-memory cache (e.g. Redis) since codes are short-lived, or a transactional database for persistence.

Back-Channel (Server-to-Server): Token Exchange

1. **Token Request:** The client's backend makes a POST to /token to exchange the code for tokens.
Example request:

```
POST /token HTTP/1.1
Host: auth.example.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=AbCdEfGhIj
&redirect_uri=https%3A%2F%2Fapp.example.com%2Fcb
&client_id=client123
&client_secret=s3cr3t          (omit for public clients, use PKCE)
&code_verifier=wK2JdE...      (if PKCE)
```

- **grant_type=authorization_code** (RFC6749) ⁹.
- Must include the same `redirect_uri`.
- **Client Authentication:** Confidential clients include `client_secret`; public clients have no secret and rely on PKCE.
- **PKCE Verification:** If `code_challenge` was provided earlier, the client now sends the original `code_verifier`. The AS hashes it (SHA-256 if `S256` was used) and compares to the stored `code_challenge` ¹⁰.

5. **Token Response:** The AS validates:

- **Code Validity:** Exists, not expired, `used=false`.
- **Client Match:** `client_id` matches stored record (prevents code theft) ¹¹.
- **Redirect URI Match:** `redirect_uri` matches exactly the stored one ⁴.
- **PKCE Check:** If applicable, computed `SHA256(code_verifier)` equals stored `code_challenge`.

- **Mark Used:** The AS marks `used=true` on the code record to prevent reuse (code replay protection).
On success, AS issues:
- **Access Token:** A JWT or opaque token for the API.
- **Refresh Token:** (Optional) a long-lived token for obtaining new access tokens.
- **ID Token:** If using OpenID Connect (beyond OAuth2), contains the user's identity (not used for API calls).
Example JSON response:

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9... (JWT)",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "Def456...",
  "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9... (JWT)"
}
```

- **Access Token Format:** Commonly a signed JWT (JWS) with claims (see below).
- **Signing:** AS signs tokens with RSA or ECDSA (e.g. RS256), publishing its public key via a JWKS endpoint (`/well-known/jwks.json`).
- **Expiration:** `exp` claim is set according to policy (e.g. 1 hour) ⁸. Refresh tokens typically live much longer.

Security Considerations

- **State (CSRF Protection):** The `state` parameter prevents cross-site request forgery. The client must verify the returned `state` matches its request ⁵.
- **Nonce (OIDC Replay Protection):** In OIDC, a `nonce` can be sent and returned in the ID Token to prevent token replay. (OIDC spec).
- **PKCE:** Ensures a stolen code cannot be used without the original `code_verifier` ¹² ⁶.
- **TLS:** All communication uses HTTPS.
- **Redirect URI Matching:** Exact match is required to avoid open redirect vulnerabilities ⁴.
- **Token Binding:** Out of scope for basic OAuth2, but stronger bindings exist in advanced use cases.

Example Code Snippet (Pseudocode)

```
# Authorization Code Generation (AS side)
def generate_auth_code(client_id, redirect_uri, user_id, scope,
code_challenge=None, method=None):
    code = base64url_encode(os.urandom(16)) # 128-bit random
    expires = now() + timedelta(seconds=60)
    db.insert("auth_codes", {
        "code": code,
```

```

        "client_id": client_id,
        "redirect_uri": redirect_uri,
        "user_id": user_id,
        "scope": scope,
        "expires_at": expires,
        "used": False,
        "code_challenge": code_challenge,
        "code_challenge_method": method
    })
    return code

# Token Exchange (AS side)
def exchange_auth_code(request):
    code = db.get("auth_codes", {"code": request.code})
    if not code or code.used or code.expires_at < now():
        return error("invalid_grant")
    if code.client_id != request.client_id:
        return error("invalid_grant")
    if code.redirect_uri != request.redirect_uri:
        return error("invalid_grant")
    if code.code_challenge:
        # Verify PKCE
        hash = sha256(request.code_verifier)
        if base64url_encode(hash) != code.code_challenge:
            return error("invalid_grant")
    # Everything valid; mark code used
    db.update("auth_codes", code.id, {"used": True})
    # Issue tokens (JWT signing omitted for brevity)
    access_token = generate_jwt(sub=code.user_id, aud=code.client_id,
scope=code.scope)
    refresh_token = generate_refresh_token()
    return {"access_token": access_token, "refresh_token": refresh_token, ...}

```

Common Errors and Debugging

- **invalid_client:** Client authentication (ID/secret) failed. Check registered credentials.
- **invalid_grant:** Code is invalid, expired, or already used. Or redirect URI mismatch.
- **invalid_request:** Missing/invalid parameters in request.
- **redirect_uri_mismatch:** The `redirect_uri` in token request doesn't match the one used in authorization request ⁴.
- **invalid_scope:** Requested scope not allowed.
- **access_denied:** User denied consent.
- **invalid_request (CSRF):** Missing or mismatched `state`.
Logs might show messages like "Auth code not found or expired", "Client redirect_uri mismatch", or "PKCE verifier failed".

Client Credentials Grant (2-Legged)

Used for **machine-to-machine** authentication (no user). The client authenticates directly with the AS to obtain an access token ¹³.

Flow

```
sequenceDiagram
    participant Client as Client (Machine)
    participant AS as Authorization Server
    participant API as Resource Server

    Client->>AS: POST /token (grant_type=client_credentials, client_id/secret, scope)
    AS->>AS: Validate client credentials
    AS-->>Client: { access_token: "...", token_type:Bearer, expires_in:3600 }
    Client->>API: GET /data (Authorization: Bearer access_token)
    API->>API: Validate token, scopes
    API-->>Client: 200 OK
```

HTTP Example

```
POST /token HTTP/1.1
Host: auth.example.com
Authorization: Basic BASE64(client_id:client_secret)
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&scope=read:api write:api
```

- **grant_type=client_credentials** (RFC6749) ¹³.
- Client must authenticate (Basic auth or form).
- Response includes an access token (no refresh token).
- Access token's `sub` is usually the client ID; scopes indicate permitted API calls.

Characteristics

- **No Refresh Token:** Usually none, since the client can just request a new token when needed.
- **Scopes:** Define what APIs the client can call (principle of least privilege).
- **Token Usage:** Sent in `Authorization: Bearer ...` header.
- **Security:** Keep client secret safe on server-side. Rotate periodically.

Refresh Token Grant

Refresh Tokens allow obtaining new access tokens without re-prompting the user. They are issued in the Authorization Code flow (if configured).

Flow

```
sequenceDiagram
    participant Client
    participant AS as Authorization Server

    Client->>AS: POST /token (grant_type=refresh_token, refresh_token=xyz, client_id=secret)
    AS->>AS: Validate refresh token & client
    AS-->>Client: { access_token: "...", expires_in:3600, refresh_token: "new_abc" }
```

HTTP Example

```
POST /token HTTP/1.1
Host: auth.example.com
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token
&refresh_token=xyzRefreshTokenValue
&client_id=client123
&client_secret=s3cr3t
```

- AS validates that the refresh token is active and bound to this client.
- Issues a new access token; may rotate the refresh token.
- Best Practice: **Refresh Token Rotation** – AS invalidates the old refresh token and issues a new one, preventing reuse of stolen tokens.

Security Considerations

- Refresh tokens are long-lived; store them securely (never expose to browser JavaScript).
- Revoke them on logout or if misuse is detected.
- Common error: `invalid_grant` if refresh token is expired or revoked.

Device Authorization Grant (RFC 8628)

The **Device Code Grant** is designed for devices with no browser or limited input (smart TVs, consoles) ¹⁴. The flow involves the user authorizing the device on a secondary device.

Sequence

```
sequenceDiagram
    participant Device
    participant UserAgent as User's Phone/Browser
    participant AS as Authorization Server

    Device->>AS: POST /device_authorization (client_id, scope)
    AS-->>Device: { device_code, user_code, verification_uri, expires_in, interval }
    Device->>Device: Display "Go to verification_uri and enter user_code"
    UserAgent->>AS: (User browses to verification_uri, inputs user_code, logs in, consents)
    AS->>AS: Authenticate user, record approval
    loop Polling
        Device->>AS: POST /token (grant_type=urn:ietf:params:oauth:grant-type:device_code, device_code)
        AS-->>Device: authorization_pending (until user completes)
    end
    AS-->>Device: { access_token: "...", refresh_token: "...", expires_in:... }
```

Details

1. **Device Request:** The device (client) calls `/device_authorization`:

```
POST /device_authorization HTTP/1.1
Host: auth.example.com
Content-Type: application/x-www-form-urlencoded

client_id=public_client_id
&scope=openid%20profile%20email
```

- AS responds with `device_code` (opaque), `user_code` (short code to show user), `verification_uri` (where user enters code), and polling details ¹⁵.

2. **User Authorization:** The user uses another device (phone/PC) to visit `verification_uri`, enters `user_code`, logs in, and consents.
3. **Device Polling:** Meanwhile, the device polls `/token` with `grant_type=urn:ietf:params:oauth:grant-type:device_code` and the `device_code`. AS returns `authorization_pending` until user completes.
4. **Token Response:** Once approved, AS returns tokens as usual.

This flow separates user login from device flow and is defined in RFC 8628 ¹⁴.

Implicit Grant (Deprecated)

The **Implicit Grant** (OAuth2) is an older 3-legged flow where the AS returns an access token directly in the redirect URI fragment (no code exchange). It is now **discouraged** due to security risks ¹. In essence:

- Client uses `response_type=token`.
- AS redirects with `#access_token=...&state=...`.
- No back-channel.

Example redirect (from OAuth.com tutorial):

```
HTTP/1.1 302 Found
Location: https://app.example.com/cb#
    access_token=2YotnFZFEjr1zCsicMWpAA
    &token_type=Bearer
    &expires_in=3600
    &state=xyzABC
```

This flow exposes tokens to the browser history and is vulnerable to interception ¹⁶. Modern best practice is to use Authorization Code with PKCE even in single-page apps ¹⁷.

Token Formats and Lifecycle

JWT Structure: Most OAuth2/OIDC tokens are JWTs:

```
{Header}.{Payload}.{Signature}
```

Example Access Token payload:

```
{
  "iss": "https://idp.example.com",
  "sub": "user789",
  "aud": ["api1", "api2"],
  "exp": 1712345678,
  "iat": 1712342078,
  "scope": "read write",
  "roles": ["admin", "finance"]
}
```

- **iss:** Token issuer (AS) ⁸.
- **sub:** Subject (user or client) ID.
- **aud:** Intended audience (API resource identifiers).

- **exp**: Expiration time.
- **scope/roles**: Authorizations granted.

ID Token (OIDC): A JWT containing user identity (e.g. email, sub, name). Used by the client to establish session, not sent to APIs.

Signing: Tokens are signed with a private key. RS256 is common. The AS publishes its public keys via a JWKS endpoint (e.g. `https://idp.example.com/.well-known/jwks.json`), so resource servers can verify signatures.

Opaque Tokens: Alternatively, tokens can be opaque strings. In that case, resource servers must call the AS's introspection endpoint (RFC 7662) to validate them.

Key Rotation: AS should periodically rotate signing keys. The JWKS endpoint may list multiple keys (with `kid` to identify) ¹².

Token Lifetimes: Typical defaults:

- **Access Token**: short-lived (e.g. 5–60 minutes).
- **Refresh Token**: long-lived (days to months).
- **Authorization Code**: very short (tens of seconds) ⁸.
- **Device Code/User Code**: short (5–15 min) ¹⁸.

Token Revocation: OAuth2 defines a revocation endpoint. Clients can revoke refresh or access tokens. Ping Federate supports token revocation APIs for force-logout.

Comparison Table: Authorization Code vs Client Credentials

Attribute	Authorization Code	Client Credentials
User Involved	Yes	No
Type	3-legged (front+back channel)	2-legged (back channel)
Tokens Issued	Access Token, Refresh Token, (ID Token)	Access Token only
Client Secret	Required (confidential); Public clients use PKCE	Required
Flow Security	High (no tokens in browser)	High (server-server)
Use Case	Web/Mobile login (with SSO) ²	Service-to-service auth
Parameters	code, redirect_uri, state, PKCE	N/A (only grant_type, scope)
Redirect URI	Must match registered	Not used
Example URI	<code>/authorize?response_type=code&...</code> ¹⁹	<i>none (direct POST)</i>

Attribute	Authorization Code	Client Credentials
Error Examples	<code>invalid_grant</code> (bad code), <code>invalid_request</code> (CSRF)	<code>invalid_client</code> (bad secret)

Production Use-Case Scenarios

1. **Web Single Sign-On:** A browser-based corporate app. Uses Authorization Code. Sequence: User clicks "Login with SSO", gets redirected to Ping Federate, logs in, then app receives an auth code and token. The app uses the ID token for session and access token for API calls.
2. **Mobile App (PKCE):** A native iOS/Android app. Uses Auth Code + PKCE (no client secret). Steps: App creates a code_verifier, derives code_challenge, opens system browser to AS with code_challenge. After login, app intercepts redirect, exchanges code with code_verifier for tokens.
3. **Backend Microservice:** A service (client ID=svcA) calls another internal API. Uses Client Credentials. It sends its client_id/secret to `/token` for an access token, then calls the API with that token (no user context).
4. **Smart TV App:** A TV app cannot show a login screen. Uses Device Flow. It polls Ping's device endpoint, shows a code and URL to the user ("Go to example.com/device and enter code"). The user uses a phone to approve. Then the TV gets a token and can access user's resources.
5. **Legacy Web App (SAML vs OAuth):** If an older web app only supports SAML, Ping can issue a SAML Assertion. For new apps, OAuth/OIDC is used. (SAML example below for contrast.)

SAML Assertion Example (Contrast)

SAML 2.0 is XML-based and used for enterprise SSO. A brief example assertion:

```
<saml:Assertion xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion" ID="_abc123"
IssueInstant="2026-04-27T12:00:00Z" Version="2.0">
  <saml:Issuer>https://idp.example.com</saml:Issuer>
  <saml:Subject>
    <saml:NameID Format="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress">user@example.com</saml:NameID>
    <saml:SubjectConfirmation Method="urn:oasis:names:tc:SAML:2.0:cm:bearer"/>
  </saml:Subject>
  <saml:Conditions NotBefore="2026-04-27T12:00:00Z"
NotOnOrAfter="2026-04-27T12:05:00Z">
    <saml:AudienceRestriction>
      <saml:Audience>https://app.example.com/sp</saml:Audience>
    </saml:AudienceRestriction>
```

```

</saml:Conditions>
<saml:AuthnStatement AuthnInstant="2026-04-27T12:00:00Z">
  <saml:AuthnContext>
    <saml:AuthnContextClassRef>urn:oasis:names:tc:SAML:
2.0:ac:classes:PasswordProtectedTransport</saml:AuthnContextClassRef>
  </saml:AuthnContext>
</saml:AuthnStatement>
<saml:AttributeStatement>
  <saml:Attribute Name="email"><saml:AttributeValue>user@example.com</
saml:AttributeValue></saml:Attribute>
  <saml:Attribute Name="role"><saml:AttributeValue>manager</
saml:AttributeValue></saml:Attribute>
</saml:AttributeStatement>
</saml:Assertion>

```

A SAML SP (service provider) would consume this assertion. In contrast, OAuth2/OIDC uses JSON (JWT) tokens and REST endpoints.

Token Validation (Resource Server Side)

When an API receives an access token, it must validate it: 1. **Signature:** Verify JWT signature using AS's public key from JWKS.

2. **Issuer (iss):** Matches expected issuer URL ⁸.
3. **Audience (aud):** Contains this API's identifier.
4. **Expiration (exp):** Not expired.
5. **Scope/Claims:** Required scopes are present.
6. **Issued At (iat) / Not Before (nbf):** Optionally check time window.

If any check fails, return `401 Unauthorized`.

For **opaque tokens**, call the AS introspection endpoint (RFC 7662) to verify activity and retrieve claims.

Common Failure Modes and Debugging

- **Invalid Redirect URI:** "redirect_uri mismatch". Happens if client's redirect URI in request doesn't exactly match the one registered. Check client config and URL encoding ⁴.
- **Invalid Grant (Auth Code):** If the code is expired, used, or doesn't match this client. AS logs like "Authorization code not found or already used".
- **Invalid Client:** Wrong client_secret or client_id. Check credentials.
- **Invalid Scope:** Requested scope isn't allowed for client. Check scope parameter.
- **Missing State:** If `state` is missing/mismatched, reject for CSRF protection.
- **Invalid Code Verifier (PKCE):** Hashed verifier doesn't match stored challenge. Use consistent S256 hashing.

- **Expired Code:** Did the code expire? (Timestamp mismatch or clock skew).
- **Access Token Rejection:** On API side, "Invalid token" means signature/audience/expiration fail. Use a JWT decoder (e.g. jwt.io) to inspect contents.

In logs, look for OAuth error codes and descriptions. E.g. Ping logs may show `OAuth [WARN] - grant_type=authorization_code failed: invalid_grant`.

Tables & Comparisons

Token Types Comparison

Token Type	Format	Contains	Lifetime	Use Case
Access Token	JWT (or Opaque)	scope, sub, aud, expiry, custom claims	Minutes	Sent to Resource APIs
ID Token	JWT	User identity (email, sub, name)	Minutes	Used by client for user auth (OIDC)
Refresh Token	Opaque/ JWT	Ties to user+client	Hours/ Days	Exchanging for new access tokens
Authorization Code	Opaque/DB	References user, client, redirect, PKCE	Seconds	Single-use code in Auth Code flow
SAML Assertion	XML	User attributes	Minutes	SAML-based SSO (enterprise apps)

Header/Parameter Examples

```
GET /authorize?response_type=code
&client_id=client123
&redirect_uri=https%3A%2F%2Fapp.com%2Fcb
&scope=openid%20profile
&state=xyzABC
&code_challenge=E9MeIhoa20wvFrEMTJguCHao
&code_challenge_method=S256
HTTP/1.1
Host: auth.example.com
```

```
POST /token HTTP/1.1
Host: auth.example.com
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=authorization_code
&code=QWE123RTY456
```

```
&redirect_uri=https%3A%2F%2Fapp.com%2Fcb
&client_id=client123
&client_secret=s3cr3t
&code_verifier=KX6hgtv1...
```

```
POST /token HTTP/1.1
Host: auth.example.com
Authorization: Basic Y2xpZW50MTIzOnMzY3IzdA==
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=client_credentials
&scope=read:api write:api
```

Further Learning and References

- **RFC 6749 (OAuth 2.0 Framework)** – sections 4.1 (Auth Code), 4.4 (Client Cred), 6 (Refresh).
- **RFC 7636 (PKCE)** – PKCE extension for Authorization Code.
- **RFC 8628 (Device Authorization)** – device flow for screenless devices.
- **RFC 8693 (Token Exchange)** – advanced grant for token swapping.
- **OpenID Connect Core** – for ID token details (beyond basic OAuth2).
- **Ping Identity Docs** – e.g. PingFederate endpoints and OAuth best practices.
- **OAuth 2.0 Simplified** (Aaron Parecki) – great explanations of flows and tokens, including code internals ⁴.

By mastering these flows and details, you'll be able to design, implement, and troubleshoot OAuth2/OIDC authentication in enterprise systems confidently.

¹ ⁴ ¹⁶ ¹⁷ The Authorization Response - OAuth 2.0 Simplified

<https://www.oauth.com/oauth2-servers/authorization/the-authorization-response/>

² ³ ⁵ ⁷ ⁸ ⁹ ¹¹ ¹³ ¹⁹ RFC 6749 - The OAuth 2.0 Authorization Framework

<https://datatracker.ietf.org/doc/html/rfc6749>

⁶ Authorization code grant with PKCE | PingAM

<https://docs.pingidentity.com/pingam/7.2/oidc1-guide/openid-connect-authorization-code-flow-pkce.html>

¹⁰ ¹² RFC 7636 - Proof Key for Code Exchange by OAuth Public Clients

<https://datatracker.ietf.org/doc/html/rfc7636>

¹⁴ ¹⁵ ¹⁸ RFC 8628 - OAuth 2.0 Device Authorization Grant

<https://datatracker.ietf.org/doc/html/rfc8628>